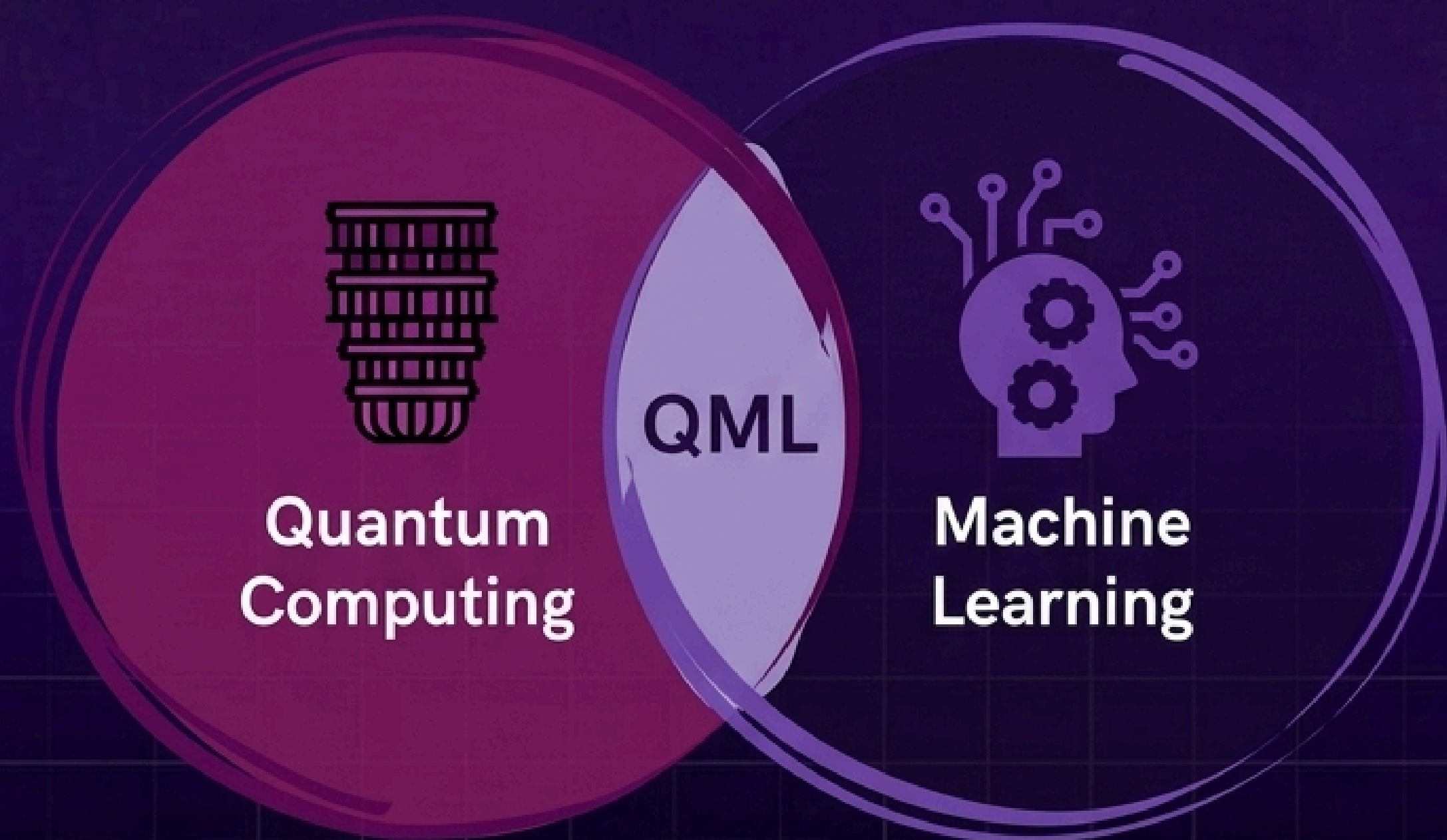


QUANTUM MACHINE LEARNING

FUNDAMENTALS



By Michael Fernandes

TABLE OF CONTENTS

01

INTRODUCTION TO QUANTUM MACHINE LEARNING

- What is QML?
- Why Quantum vs Classical ML?
- Classical ML vs QML comparison

02

QUANTUM COMPUTING BASICS

- Qubits vs Bits
- Superposition
- Entanglement
- Interference

03

QUANTUM MACHINE LEARNING WORKFLOWS

- Data Preparation
- Data Encoding
- Parameterized Quantum Circuit
- Measurement
- Loss Function (Cost Evaluation)
- Classical Optimization
- Iteration

04

CORE QUANTUM MACHINE LEARNING ALGORITHMS

- Variational Quantum Classifier (VQC)
- Quantum Kernel Methods
- QSVM (Quantum SVM)
- Quantum Neural Networks (QNN)
- Quantum PCA
- QAOA (for optimization)

05

OPTIMIZATION IN QML

- What is Optimization in QML?
- Cost Function
- Parameterized Quantum Circuits (PQC)
- Optimization Methods

06

CHALLENGES IN QML

- Hardware Challenges
- Optimization & Training Challenges
- Data & Encoding Challenges
- Measurement Challenges
- Scalability Challenges

07

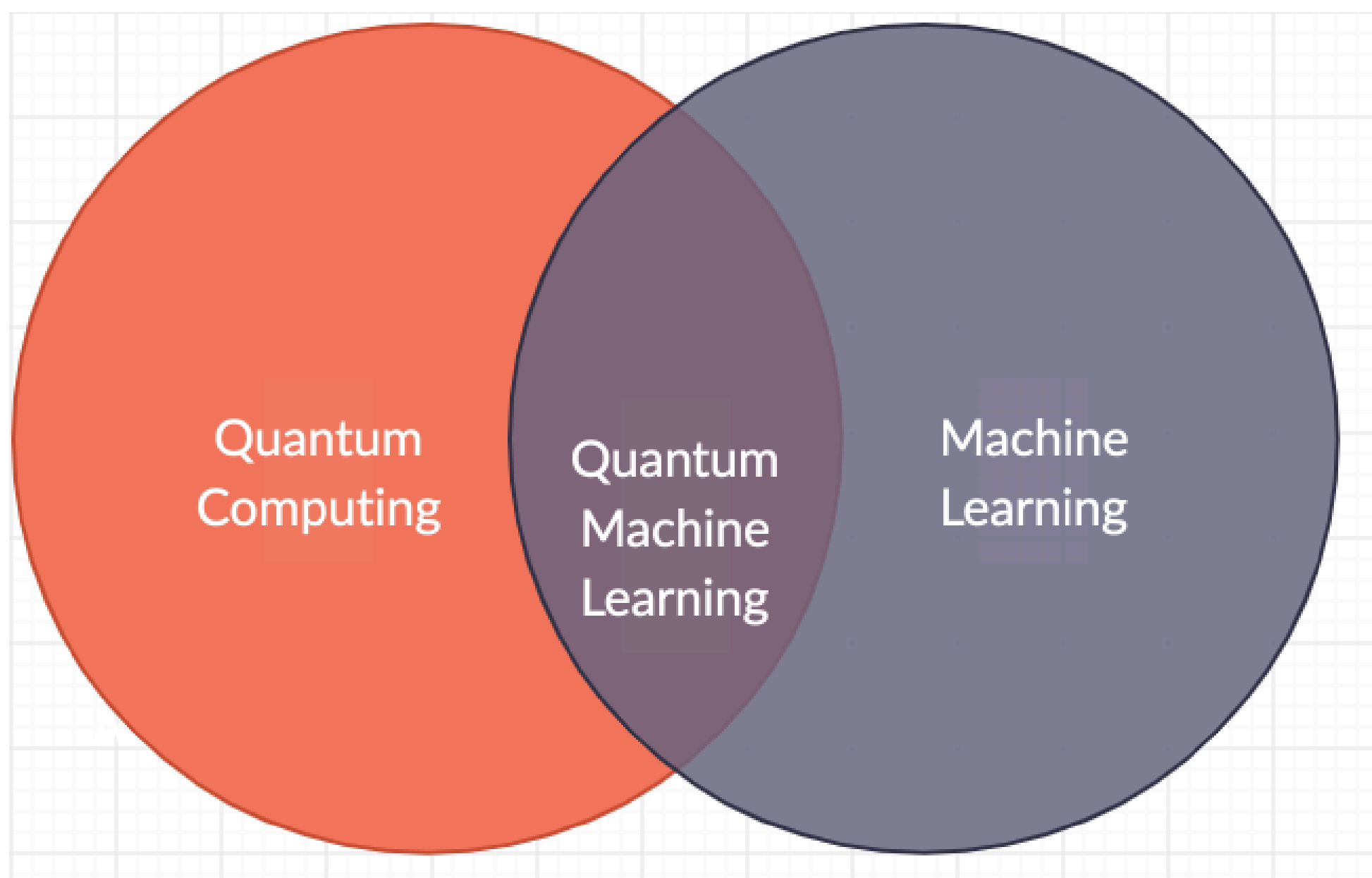
APPLICATIONS

- Finance
- Healthcare & Pharmaceuticals
- Cybersecurity
- Logistics & Supply Chain
- Manufacturing Industry
- Energy Industry

Part I - Introduction to Quantum Machine Learning

1.1

What is Quantum Machine Learning?



Quantum Machine Learning (QML) is a field that combines quantum computing and machine learning to build models that use quantum states and quantum circuits to learn patterns from data.

Instead of processing data with classical bits (0/1), QML uses:

- Qubits → can exist in multiple states at once (superposition)
- Entanglement → captures complex relationships between variables

This allows QML to represent and process information in high-dimensional quantum spaces.

Quantum ML is explored because classical ML struggles with certain types of problems:

- Extremely high-dimensional data
- Complex probability distributions
- Hard optimization landscapes

Quantum systems naturally operate in these spaces.

1. Exponential State Representation

A classical system with n bits represents 1 state at a time

An n -qubit system has a state vector of size 2^n , but extracting full information requires repeated measurements.

2. Better Feature Spaces (Quantum Advantage)

Quantum circuits map data into a Hilbert space (very high-dimensional space)

Similar to kernel methods in ML, but:

- Classical kernels : limited scalability
- Quantum kernels : potentially richer representations

3. Potential Speedups

Some quantum algorithms offer:

- Exponential speedups (rare, but powerful)
- Quadratic speedups (more realistic)

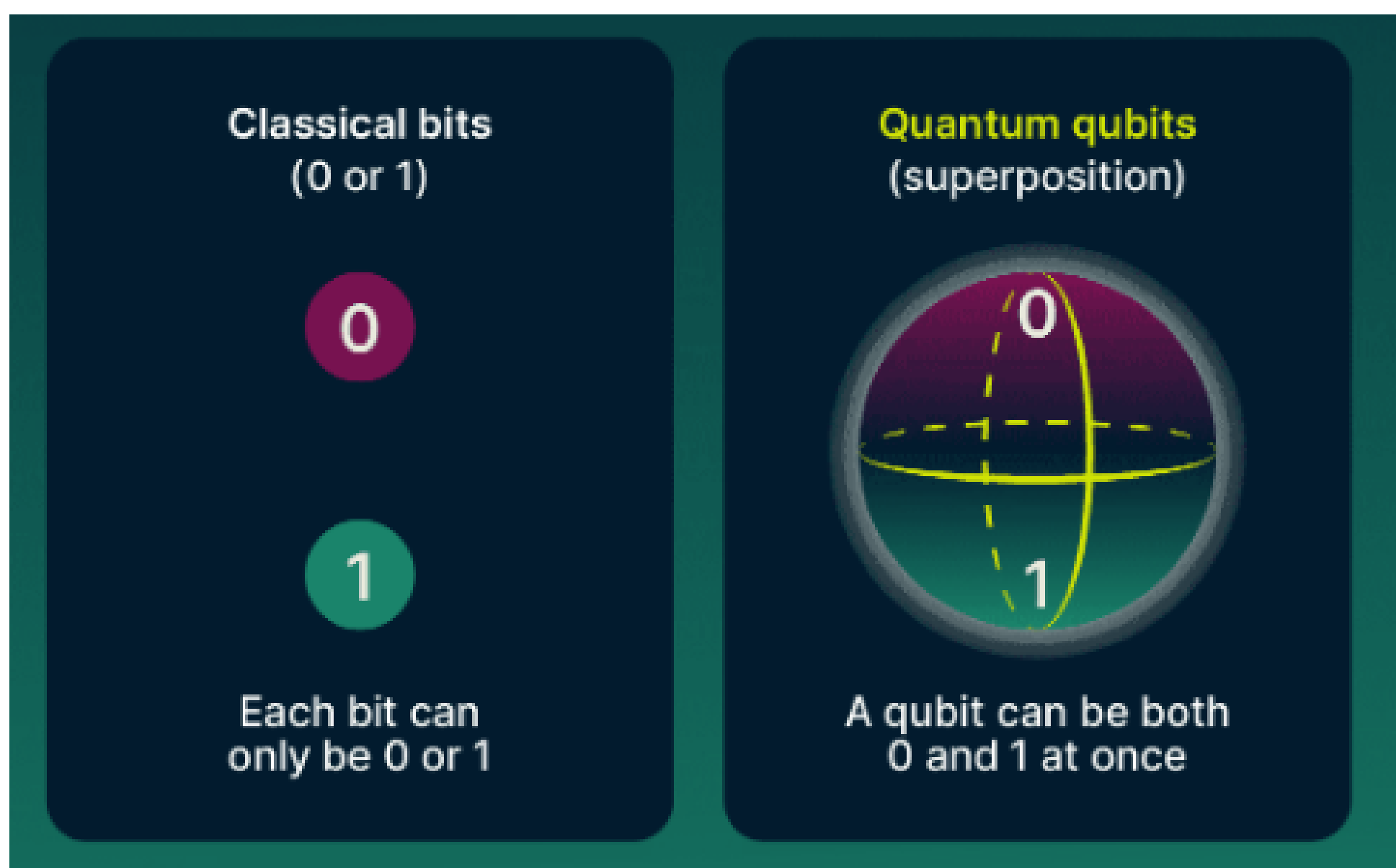
Examples:

- Search : Grover's Algorithm
- Linear systems : HHL Algorithm

Aspect	Classical ML	Quantum ML (QML)
Data Representation	Vectors, matrices	Quantum states (Hilbert space)
Basic Unit	Bit (0 or 1)	Qubit (superposition of 0 & 1)
Model Type	Neural nets, trees, SVM	Parameterized quantum circuits (PQC)
Computation	Deterministic (mostly)	Probabilistic (measurement-based)
Feature Space	Limited / engineered	Extremely high-dimensional (implicit)
Loss Evaluation	Direct computation	Expectation values of observables
Hardware	CPU / GPU / TPU	Quantum processors (NISQ devices)
Scalability	Highly scalable	Currently limited (few qubits)
Speed	Low	Potential speedups (problem-specific)
Noise Sensitivity	Low	High (quantum noise)
Maturity	Very mature	Early-stage research
Use Cases	Vision, NLP, tabular data	Optimization, quantum chemistry, finance
Interpretability	Often interpretable	Harder to interpret
Deployment	Production-ready	Experimental

Part II - Quantum Computing Basics

2.1 Qubits vs Bits



A classical bit can exist in only one state at a time—either 0 or 1, so computation is based on definite, deterministic values.

Whereas a qubit (quantum bit), on the other hand, can exist in a combination of 0 and 1 simultaneously (called superposition)

It Behaves probabilistically, allowing it to represent more information and enable more complex computations.

Definition

A qubit exists as a linear combination of basis states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

- $\alpha, \beta \rightarrow$ complex probability amplitudes

What's Actually Happening here-

- The qubit is not randomly switching between 0 and 1
- It is in a true simultaneous state until measurement

Geometric View (Bloch Sphere)

- Any qubit = a point on a sphere
- Angles define probabilities and phase

This is why quantum states are continuous, not discrete

Why It Matters

- With n qubits $\rightarrow 2^n$
- Enables exponential representation power

Entanglement is a phenomenon where multiple qubits become correlated, such that the state of one instantly determines the state of another.

Example State

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Intuition- Like two perfectly synchronized coins — if one is heads, the other is instantly heads too.

Key Property

- Cannot be described independently
- System must be treated as a whole
-

Why It Matters here

- Captures complex relationships
- Critical for multi-variable interactions
- Used in quantum algorithms and QML feature learning

Interference is the process where probability amplitudes combine, either increasing or canceling outcomes.

Conceptual Form

$$\text{Amplitude} = a_1 + a_2$$

- Constructive $\rightarrow$ amplitudes add
- Destructive $\rightarrow$ amplitudes cancel

Intuition

Like waves in water:

- In sync $\rightarrow$ bigger wave
- Out of sync $\rightarrow$ cancel out

What It Does

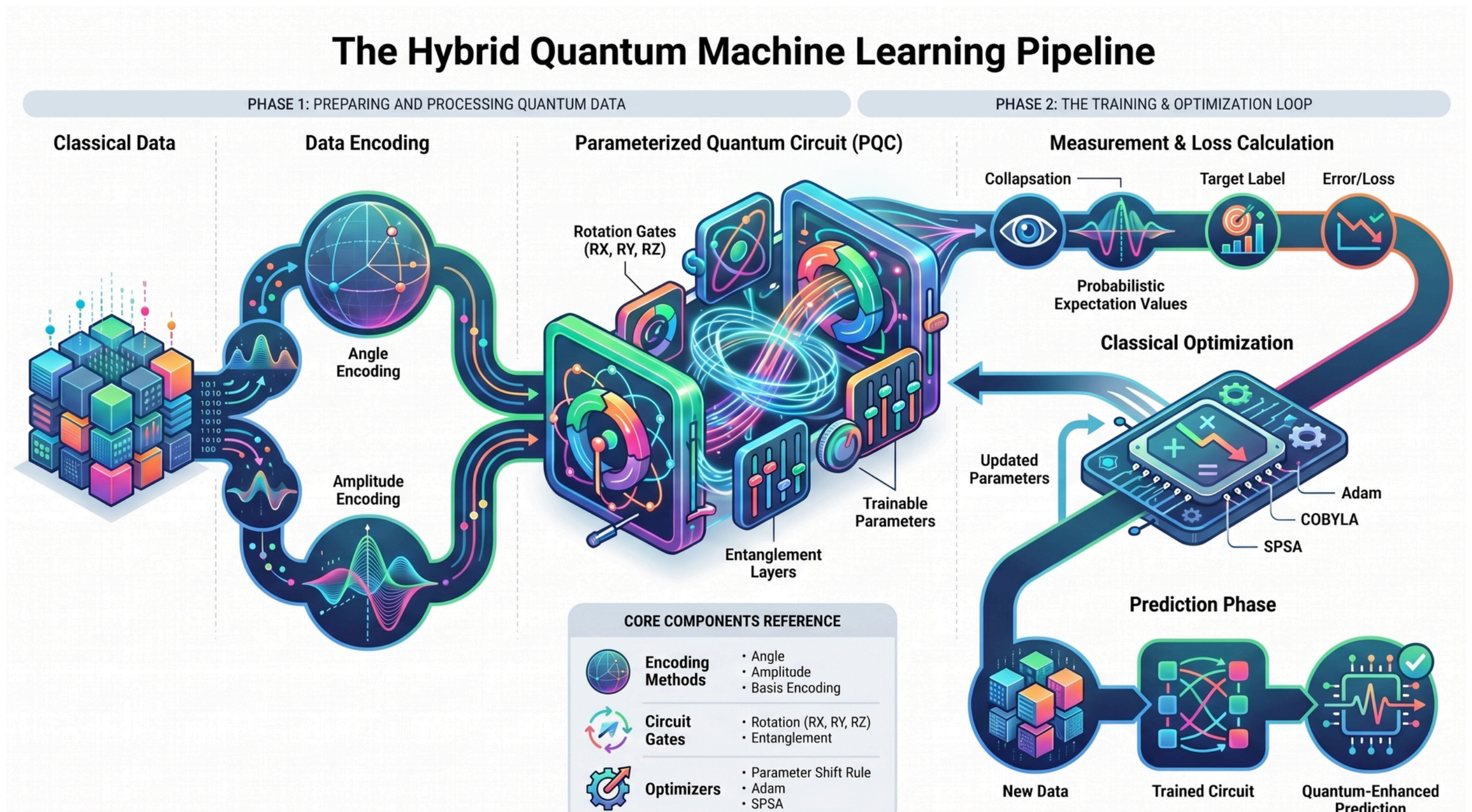
- Boosts probability of correct answers
- Suppresses incorrect ones

Why It Matters here

This is where actual computation happens

- Superposition $\rightarrow$ gives possibilities
- Entanglement $\rightarrow$ connects variables
- Interference $\rightarrow$ selects the right answer

Part III - Quantum ML Workflow



3.1

Data Preparation

- Clean missing or inconsistent data
- Normalize / scale features
- Convert data into numerical format suitable for encoding

Importance

- Quantum circuits are highly sensitive to input scaling
- Poorly scaled data can lead to ineffective encoding
- Noise in data can amplify errors due to quantum noise + measurement uncertainty

3.2

Data Encoding (Feature Mapping)

Classical data $\rightarrow$ Quantum state

$$\mathbf{x} \rightarrow |\psi(\mathbf{x})\rangle$$

Common Encoding Methods:

- Angle Encoding $\rightarrow$ map values to rotation angles
- Amplitude Encoding $\rightarrow$ encode into amplitudes
- Basis Encoding $\rightarrow$ binary representation

It Determines how well the model can learn patterns

3.3

Parameterized Quantum Circuit

Classical data $\rightarrow$ Quantum state

$$|\psi(\theta, \mathbf{x})\rangle = U(\theta)|\psi(\mathbf{x})\rangle$$

Common Encoding Methods:

- Angle Encoding $\rightarrow$ map values to rotation angles
- Amplitude Encoding $\rightarrow$ encode into amplitudes
- Basis Encoding $\rightarrow$ binary representation

It Determines how well the model can learn patterns

3.4 Measurement (Output Extraction)

Quantum state $\rightarrow$ classical value

$$\text{Output} = \langle \psi | O | \psi \rangle$$

Meaning:

- Measure observable
- Get expectation value

This acts as the model prediction

3.5 Loss Function (Cost Evaluation)

What happens:

- Compare the model's prediction with the true value
- Quantify the error using a cost (loss) function

This tells the model how wrong it is

Intuition

- Quantum circuit $\rightarrow$ produces a state
- Measurement $\rightarrow$ gives expectation value
- That value acts as the prediction/error signal

Unlike classical ML, loss is derived from quantum measurements

Common Types of Loss

- Mean Squared Error (MSE) $\rightarrow$ regression
- Cross-Entropy $\rightarrow$ classification
- Energy-based loss $\rightarrow$ variational algorithms (VQE, QAOA)

- Update the trainable parameters θ of the quantum circuit
- Aim to minimize the loss function

Methods:

- ◆ Parameter Shift Rule (Gradient Computation)

$$\frac{\partial L}{\partial \theta} = \frac{L(\theta + \frac{\pi}{2}) - L(\theta - \frac{\pi}{2})}{2}$$

- Evaluates the circuit at shifted parameters
- Used to compute gradients in quantum systems

Classical Optimizers

- Adam → adaptive learning rate
- SGD → simple gradient-based method
- COBYLA / SPSA → gradient-free (useful for noisy systems)

Process:

Repeat the following steps until convergence:

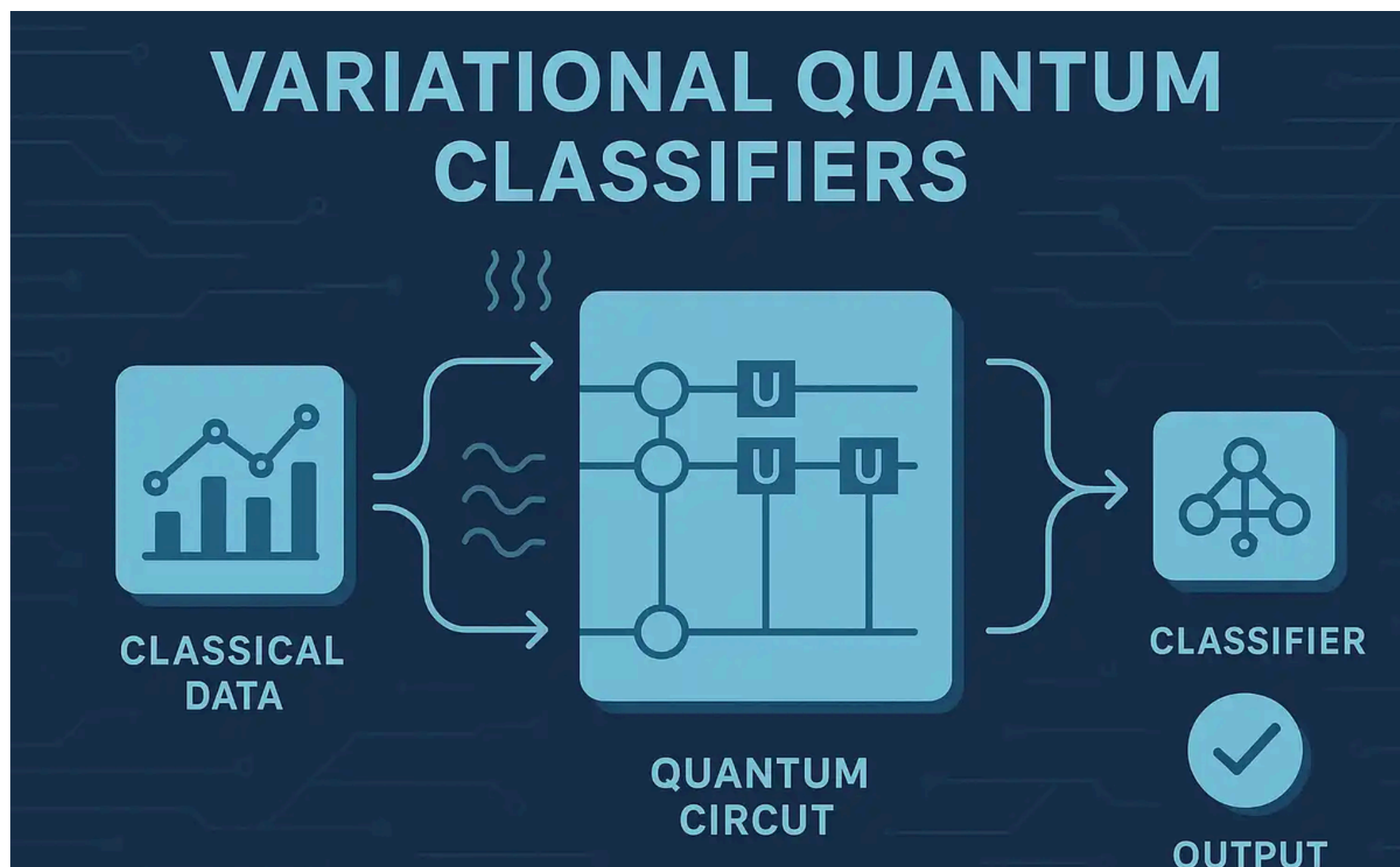
- Loss is minimized
- Model performance stabilizes

Training Loop

Run circuit → Measure → Compute loss → Update θ → Repeat

Part IV - Quantum ML Algorithms

4.1 Variational Quantum Classifier (VQC)



A **Variational Quantum Classifier** (VQC) is a hybrid quantum-classical machine learning model used for classification tasks.

It combines:

- a parameterized quantum circuit (quantum part)
- a classical optimizer (classical AI part)

The goal is to learn parameters that separate data into classes.

Think of VQC as:

“A neural network, but the hidden transformations happen inside a quantum circuit.”

Instead of neurons and weights:

- we have qubits
- and quantum gate parameters

Architecture of a VQC

1. Data Encoding (Feature Map)

Classical data must be encoded into qubits.

Example:

$$\mathbf{x} = [x_1, x_2]$$

can be encoded using rotation gates:

$$R_y(x_1), \quad R_z(x_2)$$

This creates a quantum state representing the data.

2. Variational Circuit (Ansatz)

This is the trainable quantum circuit.

Contains:

- parameterized rotation gates
- entanglement gates

Example:

$$R_y(\theta_1), \quad R_z(\theta_2)$$

where:

- θ are trainable parameters

The circuit learns patterns in data.

3. Measurement

Measure qubits.

Measurement outputs probabilities or expectation values.

Example:

$$\langle Z \rangle$$

These outputs become prediction scores.

4. Classical Optimization

A classical optimizer updates parameters.

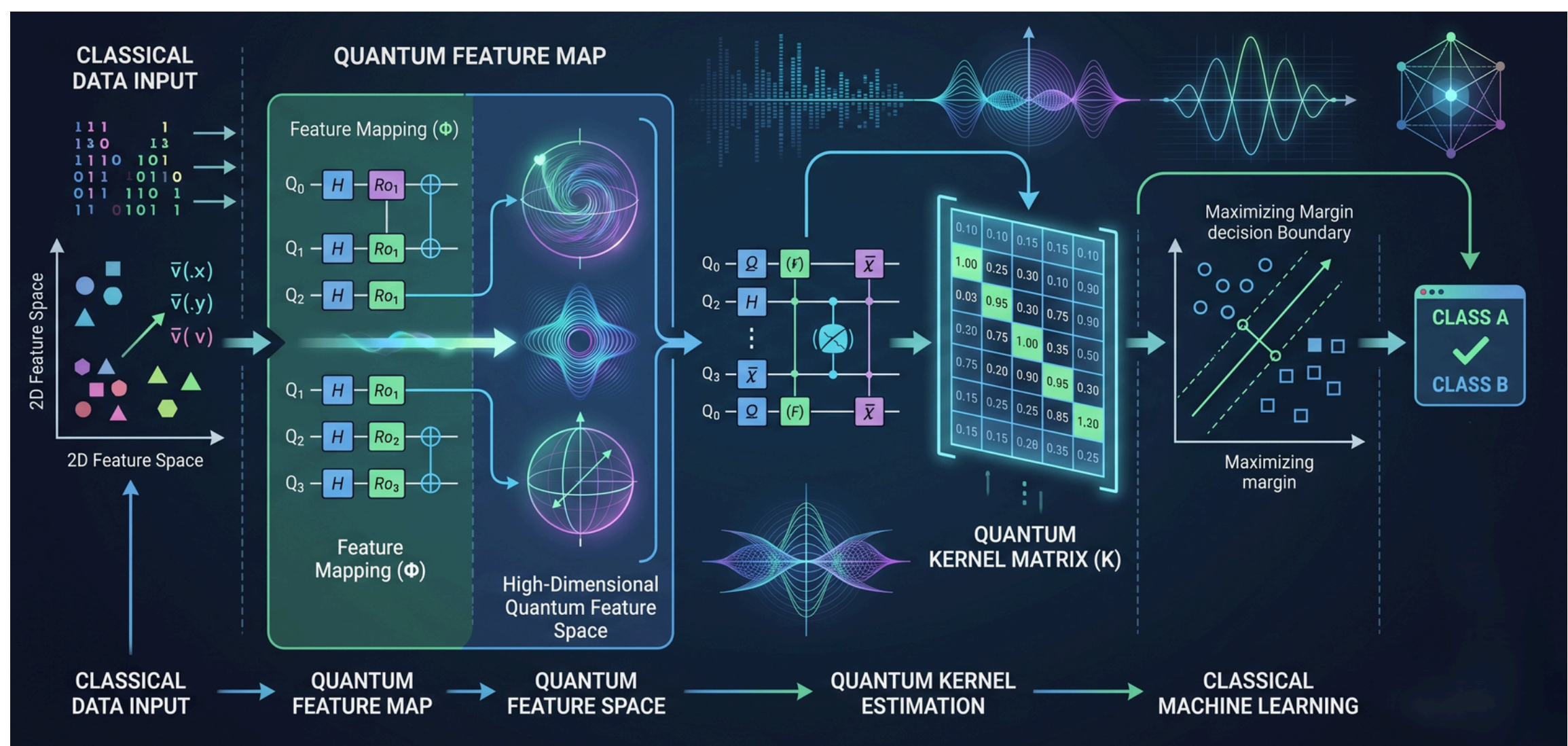
Common optimizers:

- Adam
- SGD
- COBYLA
- SPSA

Loss function:

- cross entropy
- mean squared error

◦



Quantum Kernel Methods

Quantum Kernel Methods are a class of Quantum Machine Learning (QML) algorithms that use quantum computers to compute similarity measures between data points.

The main idea:

Use a quantum computer to map classical data into a very high-dimensional quantum feature space where data may become easier to separate.

In classical ML, kernels measure similarity:

$$K(x_i, x_j)$$

Example:

- Linear kernel
- Polynomial kernel
- RBF kernel

Quantum kernel methods do this differently:

1. Encode data into quantum states
2. Compare quantum states
3. Use the similarity as the kernel

Quantum Kernel Methods Workflow

1. Classical Dataset

Start with classical data:

$$\circ \quad X = \{x_1, x_2, \dots, x_n\}$$

These may represent:

- financial features
- medical data
- images
- sensor signals

2. Data Preprocessing

Classical preprocessing is often required.

Typical steps:

- normalization
- scaling
- dimensionality reduction

Example:

$$x_i \in [0, \pi]$$

because rotation gates use angles.

3. Quantum Feature Encoding

Convert classical data into quantum states.

Feature map:

$$x \rightarrow |\phi(x)\rangle$$

This is done using a quantum circuit.

4. Quantum State Preparation

The circuit prepares:

$$|\phi(x_i)\rangle$$

for every data point.

Quantum states exist in Hilbert space.

For n qubits we have 2^n dimensional representation

5. Kernel Evaluation

Now compute similarity between two quantum states.

Kernel:

$$K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$$

This measures:

- overlap
- similarity
- closeness

6. Kernel Matrix Construction

Compute all pairwise similarities.

For n samples:

$$K_{ij} = K(x_i, x_j)$$

Kernel matrix:

	x1	x2	x3
x1	K11	K12	K13
x2	K21	K22	K23
x3	K31	K32	K33

This matrix becomes the input to classical ML algorithms.

7. Classical Machine Learning

Use the quantum kernel in classical algorithms.

Most common:

Quantum Support Vector Machine (QSVM)

Pipeline: Quantum Kernel Matrix



Classical SVM Optimization



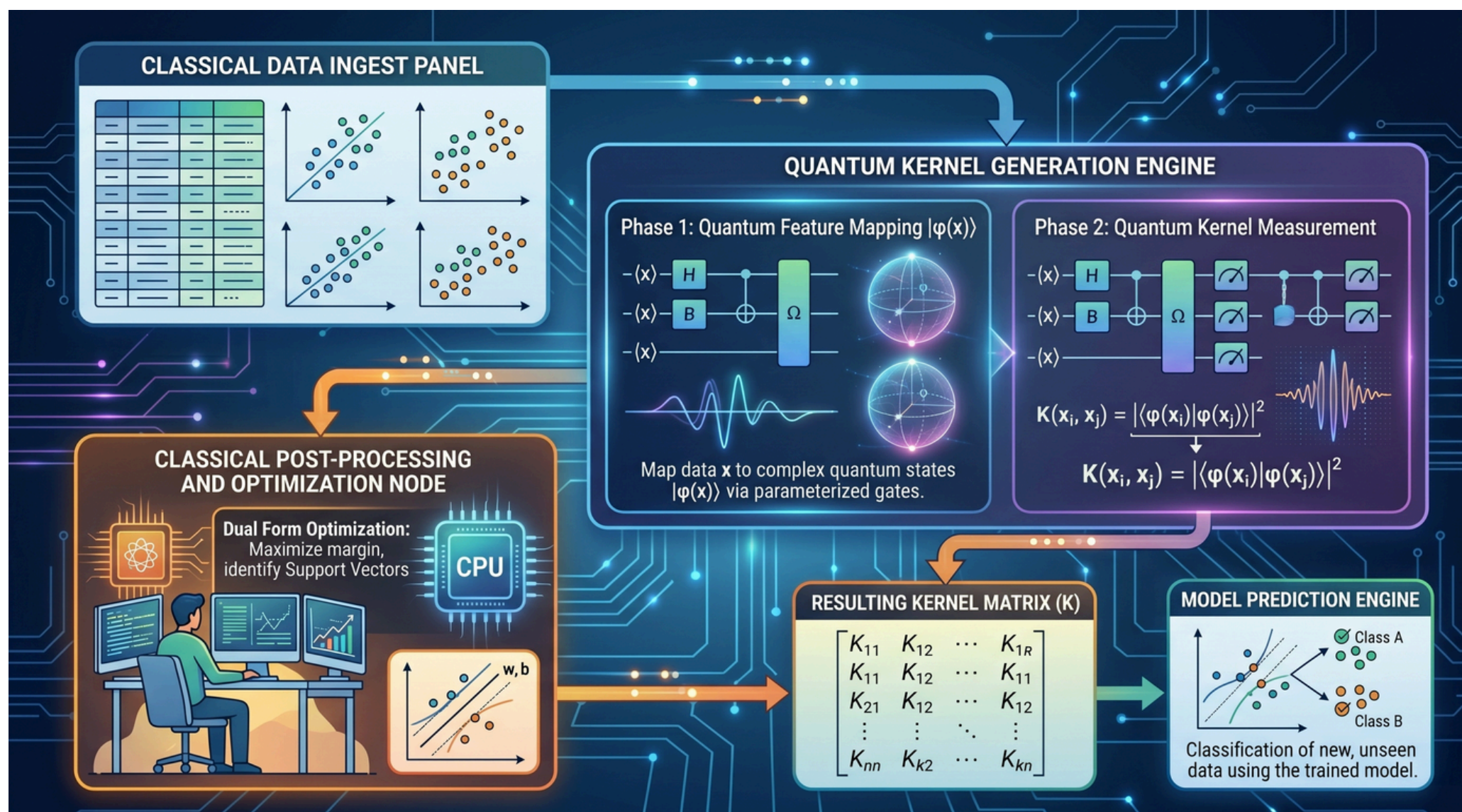
Decision Boundary

8. Prediction Phase

For a new sample x_{new}

Steps:

1. Encode into quantum state
2. Compute kernel against training points
3. Feed into trained classical model
4. Predict label



A Quantum Support Vector Machine (QSVM) is a quantum-enhanced version of the classical Support Vector Machine (SVM)

It uses a quantum computer to compute the kernel function while the optimization is still performed classically.

Classical SVMs work by:

- mapping data into feature spaces
- finding an optimal separating hyperplane

QSVM does the same thing, but:

The feature mapping is performed using quantum states

Some datasets are:

- highly nonlinear
- difficult for classical kernels

Quantum computers can create:

- exponentially large Hilbert spaces
- entangled feature representations

Potentially improving separability.

Quantum SVM Workflow

1. Classical Data Input

Example:

$x_1 = [0.5, 1.2]$

$x_2 = [1.0, 0.7]$

Labels: Class A and Class B

2. Quantum Feature Encoding

Data is encoded into qubits using gates.

Example:

$|0\rangle \text{ -- Ry}(x_1) \text{ --} \bullet \text{ --}$
 |
 $|0\rangle \text{ -- Ry}(x_2) \text{ --X--}$

This creates quantum feature states.

3. Quantum Kernel Evaluation

Compute similarity between quantum states.

Mathematically:

$$K(x_i, x_j) = |\langle 0|U^\dagger(x_i)U(x_j)|0\rangle|^2$$

where $U(x)$ = feature map circuit

4. Kernel Matrix Construction

Compute pairwise similarities:

	x1	x2	x3
x1	1	0.7	0.3
x2	0.7	1	.05
x3	0.3	0.5	1

5. Classical Optimization

Classical SVM solves optimization:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

This determines:

- support vectors
- decision boundary

6. Prediction

For a new point:

1. Encode into quantum state
2. Compute kernel with training data
3. Use trained SVM
4. Predict class

Advantages of QSVM

1. Exponential Feature Spaces

Quantum systems represent 2^n dimensional spaces using n qubits.

2. Rich Nonlinear Decision Boundaries where Entanglement creates complex correlations.

3. Less Training Than VQC - No deep variational optimization.

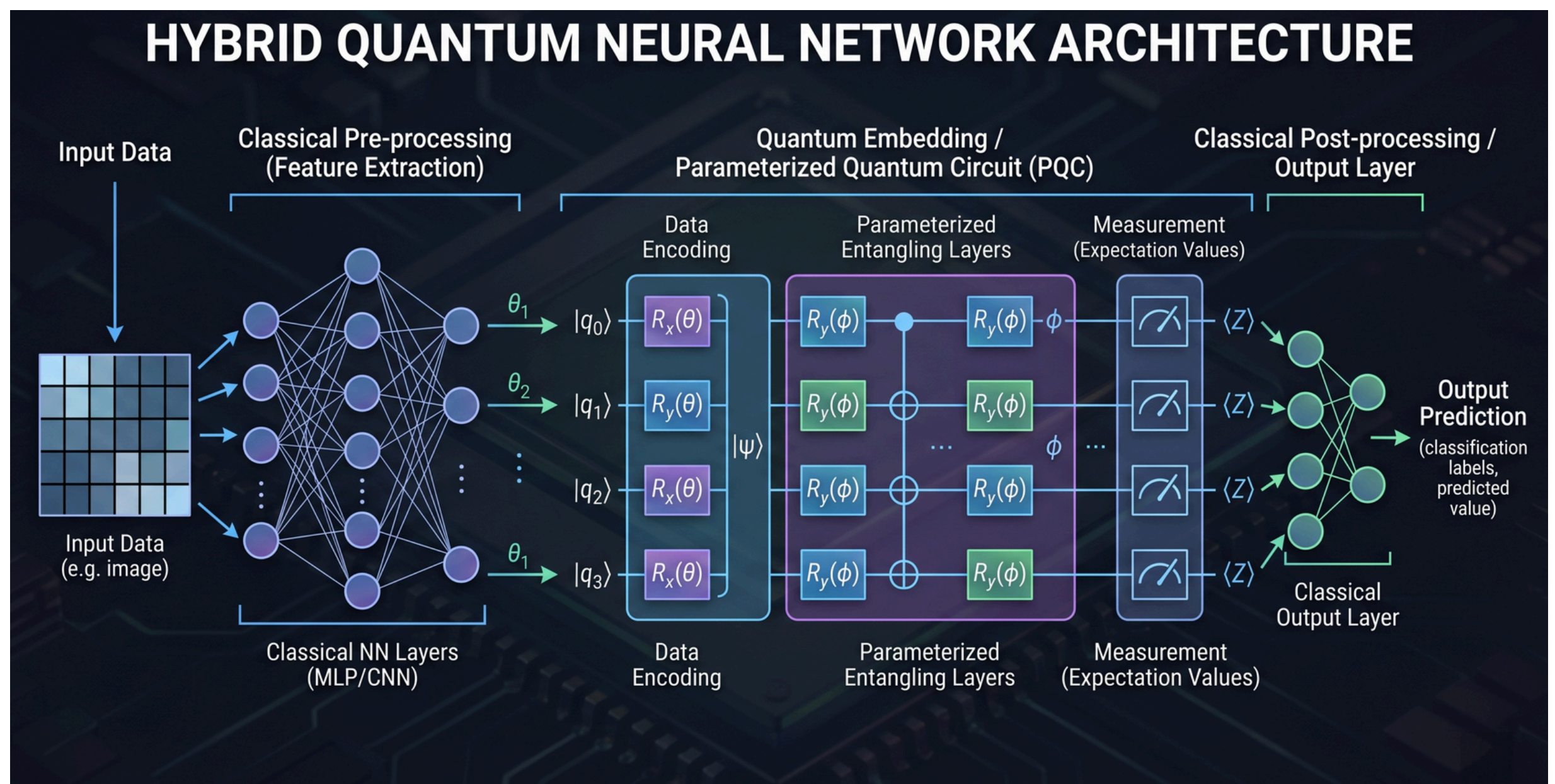
Challenges

1. Quantum Noise NISQ hardware is noisy.

2. Kernel Matrix Cost Computing all pairwise kernels scales poorly:
 $O(N^2)$

3. Data Loading Bottleneck
Encoding classical data is expensive.

4. Scalability
Limited qubits and circuit depth.



A Quantum Neural Network (QNN) is a machine learning model that uses:

- quantum circuits
- parameterized quantum gates
- quantum measurements

to learn patterns from data.

It is the quantum analogue of a classical neural network.

Classical neural networks use:

- neurons
- weights
- activation functions
-

QNNs use:

- qubits
- trainable quantum gates
- measurements

Quantum Neural Network Architecture

1. Input Layer

Classical data: $x=[x_1, x_2, \dots, x_n]$

Examples:

- stock prices
- images
- sensor values
- medical data

2. Quantum Data Encoding Layer

Classical data must be embedded into qubits.

This is called Quantum Feature Encoding or Quantum Embedding

Common Encoding Methods

- 1.Angle Encoding - Features as rotation angles
- 2.Amplitude Encoding - Features stored in amplitudes
- 3.Basis Encoding - Binary states
- 4.Data Re-uploading - Multiple embeddings

3. Entanglement Layer

Entanglement allows qubits to interact.

Common gates:

- CNOT
- CZ
- Controlled rotations

Entanglement enables:

- richer representations
- high-dimensional correlations
- quantum advantage potential

4. Measurement Layer

Quantum states cannot be directly read fully.
Instead:

- qubits are measured
- expectation values are extracted

Example - $\langle Z \rangle$

Measurement outputs become:

- probabilities
- logits
- regression values

5. Classical Postprocessing

Measurement results are processed classically.

Examples:

- softmax
- sigmoid
- thresholding

Loss Function

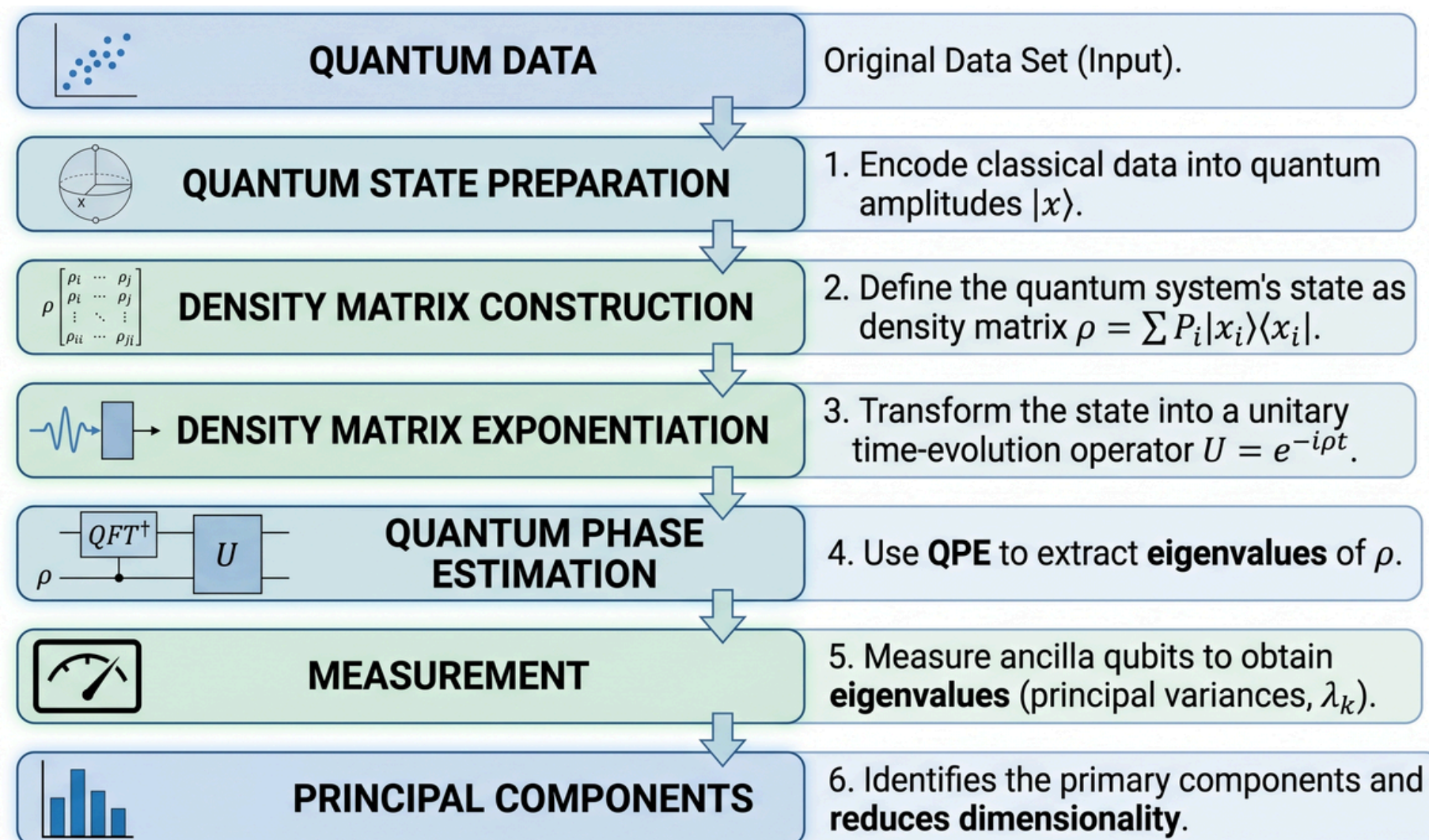
Compare prediction with actual labels.

Examples:

- Cross entropy
- Mean squared error

Types of QNN Architectures

1. Variational Quantum Circuit (VQC) - Uses Parameterized quantum gates.
2. Quantum Convolutional Neural Networks (QCNN) - Used for images.
3. Quantum Recurrent Neural Networks (QRNN) - Used in Sequential/time-series learning.
4. Hybrid Quantum-Classical Networks - Used in practical QML systems.



Quantum PCA is a quantum algorithm designed to find the:

- principal components
- dominant eigenvectors
- major variance directions

of a dataset using quantum computation.

It is the quantum analogue of Principal Component Analysis (PCA) from classical machine learning.

Classical PCA:

- reduces dimensionality
- compresses data
- finds important patterns

Quantum PCA aims to:

- extract dominant eigenvectors/eigenvalues
- analyze quantum states
- compress quantum information

Potentially faster for large quantum systems.

Quantum PCA Architecture

1. Quantum Data Encoding Layer

Classical or quantum data is encoded into quantum states.

Input states:

$|x_1\rangle, |x_2\rangle, \dots$

2. Density Matrix Construction

Construct

$$\rho = \frac{1}{M} \sum_i |x_i\rangle\langle x_i|$$

This acts similarly to covariance matrix in classical PCA

The density matrix captures:

- correlations
- probability structure
- variance information

3. Density Matrix Exponentiation Layer

Central qPCA component. It implements $e^{-i\rho t}$

Quantum Phase Estimation requires a unitary operator.

Exponentiation converts:

- density matrix
- into
- unitary evolution

4. Quantum Phase Estimation (QPE) Module

This is the core eigenvalue extraction system.

If:

$$\rho|u_i\rangle = \lambda_i|u_i\rangle$$

then QPE estimates λ and associated eigenvectors

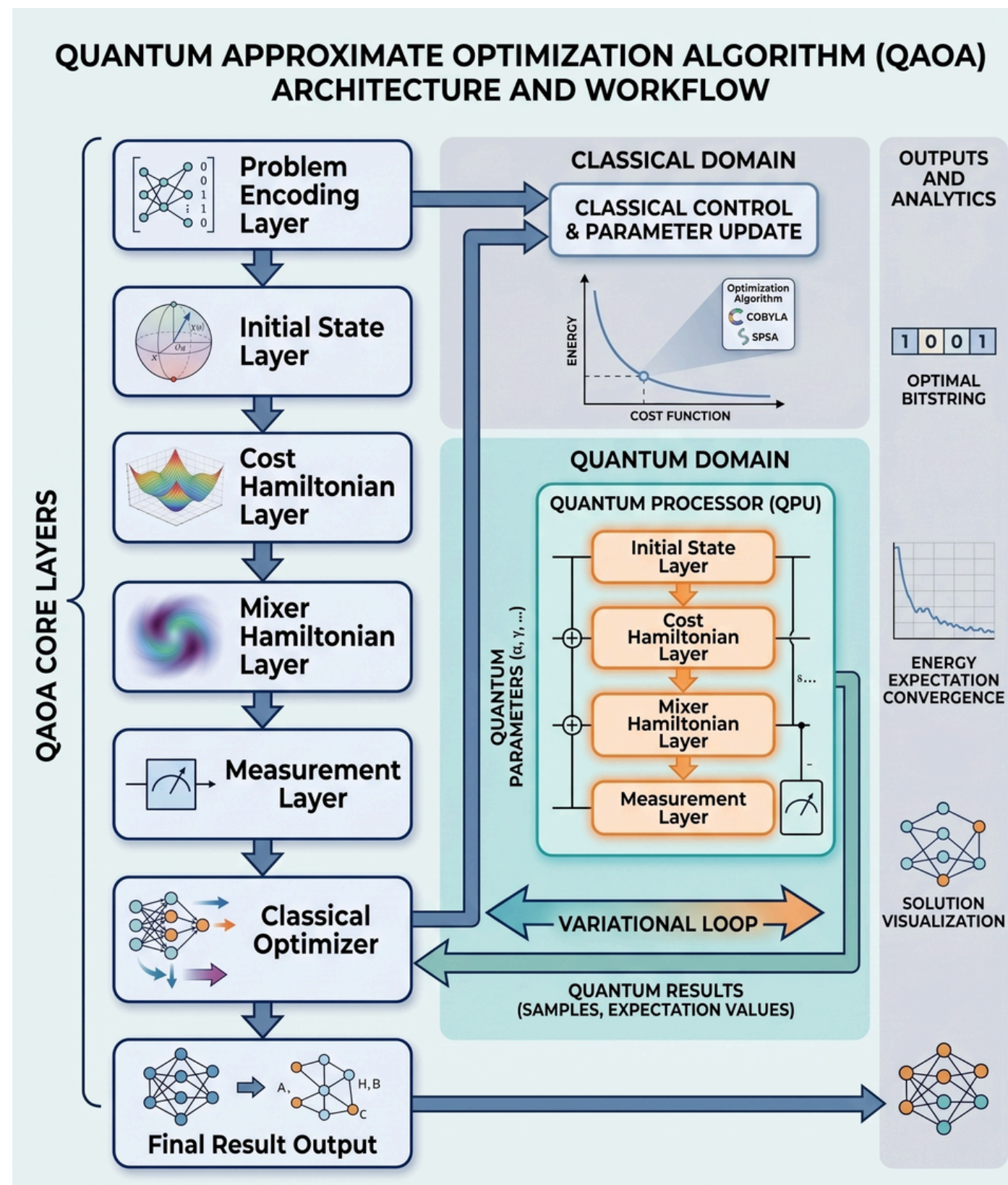
5. Measurement Layer

Measurement extracts:

- eigenvalues
- dominant eigenvectors

These correspond to:

- principal components
- major variance directions



The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical algorithm used for solving:

- combinatorial optimization problems
- constraint satisfaction problems
- graph optimization problems

It is one of the most important algorithms in the NISQ (Noisy Intermediate-Scale Quantum) Era

QAOA tries to find the minimum (or maximum) of a cost function using a parameterized quantum circuit.

It alternates between:

1. Problem Hamiltonian
2. Mixer Hamiltonian

while optimizing parameters classically.

1. Optimization Problem Encoding

- First convert the optimization problem into a:
- Cost Hamiltonian
- CCCThis mathematically represents the objective function.

Example: Max-Cut Hamiltonian

$$C = \sum_{(i,j) \in E} \frac{1}{2} (1 - Z_i Z_j)$$

where:

- Z_i = Pauli-Z operator
- graph edges define interactions

2. Initial State Preparation Layer

Initialize all qubits into superposition.

Initial state:

$$|s\rangle = |+\rangle^{\otimes n}$$

3. Cost Hamiltonian Layer

Apply:

$$U(C, \gamma) = e^{-i\gamma C}$$

It is used to encode optimization objective into quantum phases

4. Mixer Hamiltonian Layer

Apply:

$$U(B, \beta) = e^{-i\beta B}$$

Typical mixer Hamiltonian:

$$B = \sum_i X_i$$

The mixer:

- explores the search space
- transitions between candidate solutions
- prevents stagnation

Equivalent to:

- exploration in optimization

6. Measurement Layer

Measure qubits after all layers.

Each bitstring corresponds to a candidate solution

7. Classical Optimization Layer

Compute expected cost:

$$\langle C \rangle = \langle \gamma, \beta | C | \gamma, \beta \rangle$$

Significance of Each Layer

1. Hadamard Layer - Generate all possibilities
2. Cost Layer - Favor good solutions
3. Mixer Layer - Explore alternatives
4. Measurement Layer - Sample solutions

Part V - Optimization in QML

5.1

What is Optimization in QML?

Optimization in QML is the process of finding the best quantum circuit parameters that minimize a cost function.

It is the learning mechanism behind most quantum machine learning algorithms.

Quantum machine learning models contain:

- parameterized quantum gates
- trainable angles/parameters
- quantum circuits

Optimization adjusts these parameters so the model performs better.

The main goal is to find Optimal Parameters

$$\theta^* = \arg \min_{\theta} L(\theta)$$

where:

- θ = trainable quantum parameters
- $L(\theta)$ = loss function

Optimization in QML is similar to training a neural network.

- Classical ML update weights
- Quantum ML update quantum gate parameters

Suppose a quantum circuit contains:

$R_y(\theta_1), R_z(\theta_2)$

These rotation angles are trainable.

Optimization changes:

- θ_1
- θ_2

until the circuit gives better predictions.

A Cost Function (also called a Loss Function or Objective Function) measures: how good or bad a quantum machine learning model is performing. It guides the optimization process during training.

The quantum model makes predictions.

The cost function compares Prediction vs True Output and produces an error value.

Optimization tries to:

$$\min L(\theta)$$

where:

- $L(\theta)$ = cost function
- θ = trainable quantum parameters

Lower cost means:

- better predictions
- better learning

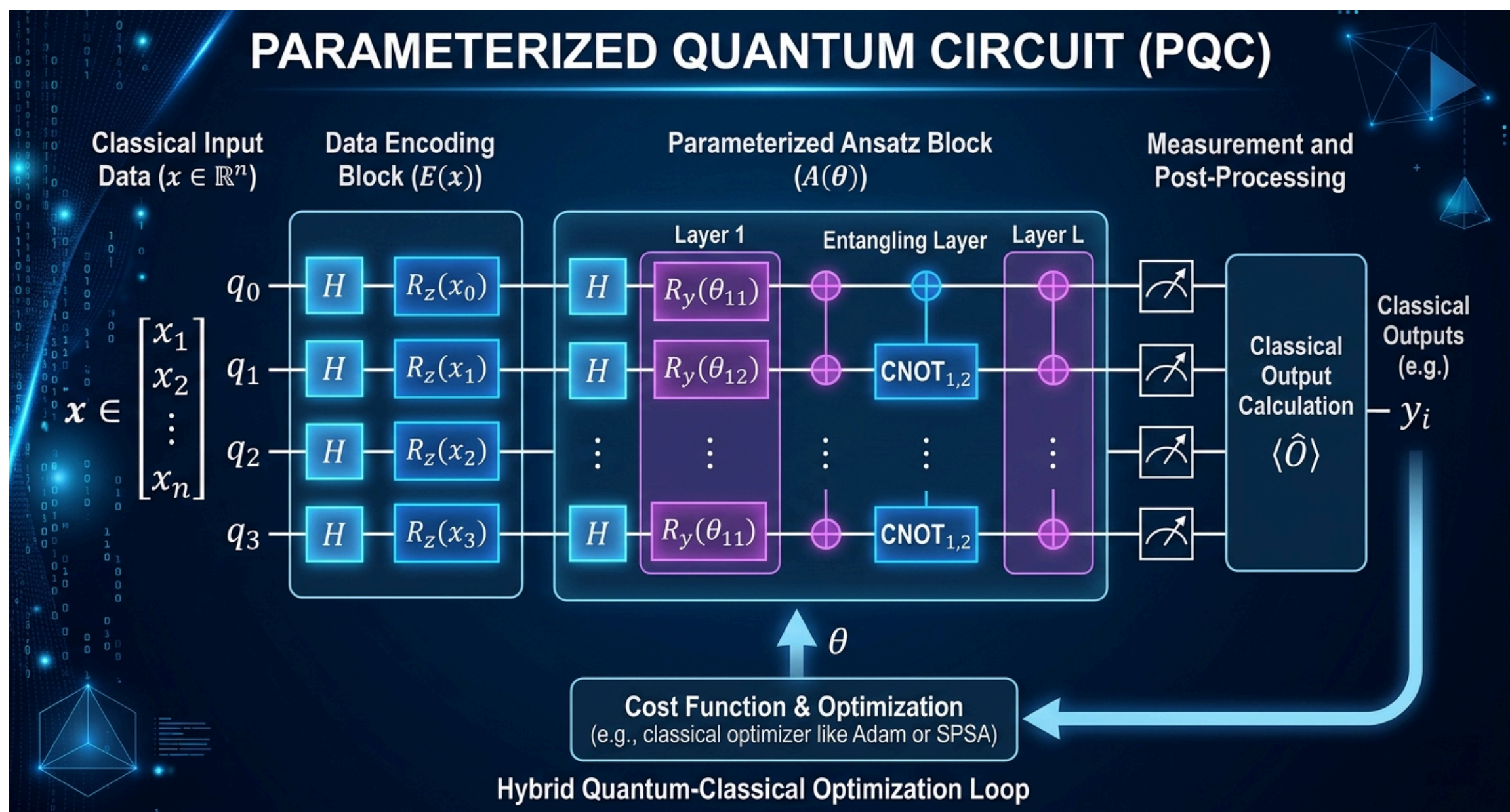
Without a cost function:

- the model cannot learn
- parameters cannot improve
- optimization has no target

General Cost Function Form

where: $L(y, \hat{y})$

- y = true value
- $\hat{y}$ = predicted value



A Parameterized Quantum Circuit (PQC) is a quantum circuit that contains:

- trainable quantum gates
- adjustable parameters
- tunable rotation angles

These parameters are optimized during training.

PQCs are the foundation of:

- Quantum Neural Networks (QNNs)
- Variational Quantum Circuits (VQCs)
- VQE
- QAOA
- many QML algorithms

A PQC acts like a:

Trainable Quantum Model

Instead of learning:

- neural network weights

it learns:

- quantum gate parameters

1. Classical Input Data

On the left side: $x \in \mathbb{R}^n$

This means classical input data:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Examples:

- Image pixels
- Financial features
- Sensor readings
- Text embeddings
- Stock indicators

Example:

$x=[0.2, 1.5, -0.7, 0.9]$

The quantum computer cannot directly process classical vectors.

2. Data Encoding Block $E(x)$

This converts classical data into a quantum state.

The diagram uses:

- Hadamard gates H
- Rotation gates $R_z(x_i)$

Hadamard Gate H

The Hadamard gate creates superposition.

$$H |0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

This means:

A qubit becomes both 0 and 1 simultaneously.

Rotation Encoding

The data is encoded into rotation angles:

$$R_z(x_i) = e^{-ix_iZ/2}$$

Each feature value rotates the qubit by an amount determined by x_i .

Example:

- x_0 rotates qubit q_0
- x_1 rotates qubit q_1

This is called:

- Angle Encoding
- Rotation Encoding
- Feature Map

Encoding determines:

- how information enters the quantum system
- expressive power
- separability of data
- quantum advantage potential

3. Parameterized Ansatz Block $A(\theta)$

This is the “learnable” part of the quantum model.

The diagram shows:

- multiple layers
- trainable rotation gates
- entangling gates

This is analogous to hidden layers in neural networks.

An ansatz is:

A parameterized quantum circuit architecture chosen to learn patterns.

Examples:

- Hardware-efficient ansatz
- Strongly entangling ansatz
- QAOA ansatz
- UCC ansatz (chemistry)

4. Entangling Layer

The blue CNOT gates create entanglement.

Example:

CNOT_{1,2}

This connects qubits.

Entanglement allows:

- correlation between qubits
- complex feature interactions
- quantum advantage
- non-classical representations

Without entanglement:

- qubits behave independently
- model capacity is limited

5. Measurement

After quantum operations:

the quantum state is measured.

The measurement symbols on the right indicate Quantum → Classical conversion

Quantum states collapse during measurement.

The diagram shows $\langle \hat{O} \rangle$

This means expectation value of observable $\hat{O}$

$$\langle \hat{O} \rangle = \langle \psi | \hat{O} | \psi \rangle$$

This is the final numerical output from the quantum circuit.

Examples:

- probability
- class prediction
- energy estimate
- regression value

6. Classical Output Calculation

The measurement results become classical outputs y_i
These outputs are used to compute Loss Function

5.4

Optimization Methods in QML

In Quantum Machine Learning, optimization methods are used to train a Parameterized Quantum Circuit (PQC) by updating its parameters:

$\theta=(\theta_1,\theta_2,...,\theta_n)$

Goal:

$$\min_{\theta} L(\theta)$$

- $L(\theta)$ = cost/loss function
- θ = trainable quantum gate parameters

Method	Algorithm
Gradient-Based	SGD, Adam, RMSProp
Gradient-Free	COBYLA, Nelder-Mead
Stochastic	SPSA
Quantum-Aware	Quantum Natural Gradient
Evolutionary	Genetic Algorithms
Bayesian	Bayesian Optimization

1. Gradient Descent

Basic optimization method.

Updates parameters using gradients.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

It Moves parameters in the direction that reduces loss.

Pros

- Simple
- Easy to implement

Cons

- Can be slow
- May get stuck

2. Adam Optimizer

The full form of Adam is Adaptive Moment Estimation

it Combines

- momentum
- adaptive learning rate

It Makes training faster and more stable.

Pros

- Fast convergence
- Handles noisy gradients well
- Works well for deep PQCs

Cons

- More hyperparameters

3. SGD (Stochastic Gradient Descent)

Uses approximate/noisy gradients.

Idea

Updates parameters using small random samples.

Pros

- Memory efficient
- Simple

Cons

- Noisy updates
- Can oscillate

4. COBYLA

Gradient-free optimizer.

Full form is Constrained Optimization BY Linear Approximation

It Searches parameter space without computing gradients.

Pros

- Good for noisy quantum hardware
- Stable for small circuits

Cons

- Slow for large problems

5. SPSA

Very popular in quantum computing.

Full form Simultaneous Perturbation Stochastic Approximation

It Estimates gradients using only two circuit evaluations.

Pros

- Hardware efficient
- Noise tolerant
- Scales well

Cons

- Approximate gradients
- Can be unstable

6. RMSProp

Adapts learning rates based on recent gradients.

Large gradients → smaller steps

Small gradients → larger steps

Pros

- Better convergence than SGD

Cons

- Sometimes unstable

7. Momentum Optimizer

Adds velocity to gradient descent.

It Builds speed in consistent directions.

Pros

- Faster convergence
- Reduces oscillation

Cons

- Can overshoot minima

8. Nelder-Mead

Gradient-free search method.

It Uses a geometric simplex to search for minima.

Pros

- Works for noisy systems

Cons

- Poor scalability

9. Quantum Natural Gradient (QNG)
Quantum-aware optimizer.

It Uses quantum geometry for smarter updates.

$$\theta_{t+1} = \theta_t - \eta F^{-1} \nabla L$$

Pros

- Better convergence
- Efficient optimization paths

Cons

- Computationally expensive

Optimizer	Type	Best Use
Adam	Gradient-based	General QML training
COBYLA	Gradient-free	Noisy hardware
SPSA	Stochastic	Real quantum devices
SGD	Gradient-based	Simple models
QNG	Quantum-aware	Advanced QML

Part VI - Challenges in QML

6.1

Hardware Challenges

Problems related to quantum devices and physical qubits.

a) Noise

Quantum computers are highly noisy.

- Incorrect outputs
- Unstable training
- Poor accuracy

b) Decoherence

Qubits lose quantum information quickly.

- Superposition collapses
- Entanglement is destroyed
- Long circuits fail

c) Limited Qubit Count

Current devices have very few reliable qubits.

- Small models only
- Limited scalability
- Restricted feature representation

d) Connectivity Constraints

Not all qubits are directly connected.

- Increased circuit depth
- More noise

e) Lack of Error Correction

Fault-tolerant quantum computing is still immature.

Effect

- QML mostly runs on noisy hardware
- Large reliable models are difficult

Problems during PQC training.

a) Barren Plateaus

Gradients vanish exponentially.

$$\nabla L(\theta) \rightarrow 0$$

Effect

- Training stops
- Parameters stop updating

b) Difficult Optimization Landscapes

Loss surfaces can contain:

- Local minima
- Flat regions
- Sharp oscillations

Effect

- Slow convergence
- Unstable optimization

c) Noisy Gradients

Hardware noise corrupts gradient estimates.

Effect

- Wrong parameter updates
- Unstable learning

d) Expensive Circuit Evaluations

Training requires many circuit executions.

Effect

- High computational cost
- Slow learning

Problems in handling classical data.

a) Quantum Data Encoding Bottleneck

Classical data must be converted into quantum states.

Methods:

- Angle encoding
- Amplitude encoding
- Basis encoding

Effect

- Encoding can be computationally expensive

b) Large Dataset Handling

Quantum hardware struggles with large datasets.

Effect

- Limited training scale
- High memory requirements

c) Dataset Limitations

Few benchmark datasets exist for QML.

Effect

- Difficult performance evaluation
- Limited real-world testing

6.4

Measurement Challenges

a) Shot Complexity

Quantum outputs are probabilistic.

Need repeated measurements:

Effect

- Increased runtime
- Expensive computation

b) Readout Errors

Measurement devices may return incorrect states.

Effect

- Reduced model reliability

6.5

Scalability Challenges

Problems when increasing model size.

a) Circuit Depth Growth

Larger models require deeper circuits.

Effect

- More noise
- Harder optimization

b) Exponential State Space

Quantum systems grow exponentially.

State Space Size= 2^n

Effect

- Hard classical simulation
- Resource explosion

c) Computational Cost

Training large PQCs is expensive.

Effect

- Slow experiments
- High hardware usage

Part VII - Applications Of QML



7.1

Finance

One of the biggest application areas of QML.

a) Portfolio Optimization

Finding the best asset allocation under constraints.

b) Risk Analysis

Modeling market uncertainty and financial risk.

c) Fraud Detection

Detecting abnormal transaction patterns.

d) Option Pricing

Quantum-enhanced stochastic simulations.

e) Algorithmic Trading

Pattern recognition in financial markets.

7.2

Healthcare & Pharmaceuticals

QML can help analyze complex biological systems.

a) Drug Discovery

Predicting molecular interactions faster.

b) Protein Folding

Studying molecular structures.

c) Medical Diagnosis

Quantum-enhanced pattern recognition.

d) Genomics

Analyzing high-dimensional genetic data.

7.3

Cybersecurity

Quantum-enhanced AI for secure systems.

a) Intrusion Detection

Detecting cyberattacks using anomaly detection.

b) Cryptography

Quantum-safe encryption research.

c) Threat Intelligence

Advanced attack pattern analysis.

QML may improve:

- anomaly detection
- pattern matching
- security optimization

7.4

Logistics & Supply Chain

Optimization-heavy industry where QML is promising.

a) Route Optimization

Finding shortest and most efficient paths.

b) Warehouse Optimization

Efficient inventory placement.

c) Supply Chain Prediction

Demand forecasting and scheduling.

d) Traffic Optimization

Smart transportation systems.

7.5

Manufacturing Industry

QML helps optimize industrial systems.

a) Predictive Maintenance

Detecting machine failures before breakdowns.

b) Quality Control

Defect detection using quantum-enhanced vision.

c) Process Optimization

Improving industrial efficiency.

7.6

Energy Industry

Energy systems involve large-scale optimization.

a) Smart Grid Optimization

Efficient electricity distribution.

b) Energy Demand Forecasting

Predicting future energy consumption.

c) Battery Material Discovery

Finding better energy storage materials.